

Spécifications fonctionnelles

Pourquoi chaque fonction existe,
et quelle formule elle applique

Ce document parcourt les 20 fichiers Python du projet et, pour chaque fonction, répond à deux questions : à quoi sert-elle dans la chaîne de calcul, et quelle est la formule sous-jacente lorsqu'elle en applique une. Les références d'articles renvoient au règlement délégué (UE) 2015/35, sauf mention contraire.

Table des matières

1 Conventions de lecture	2
2 Package scr_data — fabrication des données	2
2.1 config.py — hypothèses et aléa	2
2.2 courbe.py — courbe des taux sans risque	2
2.3 mortalite.py — tables de mortalité	3
2.4 actifs.py — inventaire ligne à ligne	3
2.5 passif_vie.py — model points	4
2.6 non_vie_sante.py — triangles et santé SLT	4
2.7 bilan.py et export.py	5
3 generer_donnees.py	5
4 Package scr — moteur de calcul	5
4.1 passif.py — best estimate déterministe	5
4.2 marche.py — risque de marché	6
4.3 contrepartie.py	7
4.4 souscription.py — vie et santé	8
4.5 non_vie.py	8
4.6 agregation.py	9
4.7 esg.py — générateur de scénarios	9
4.8 alm.py — projection actif-passif	9
4.9 reforme.py — régime 2027	10
4.10 marge_risque.py	10
4.11 modele_interne.py	10
4.12 orsa.py	11
5 Scripts d'extraction	11
6 Scripts de calcul	12
7 Restitution	12
8 Index : article du règlement, fonction correspondante	13

1 Conventions de lecture

Chaque entrée donne la signature de la fonction, la raison de son existence, puis la formule quand il y en a une. Les fonctions dont le nom commence par un tiret bas sont internes : elles n'ont de sens qu'appelées depuis leur module.

Notations communes : s_t taux zéro-coupon de maturité t , $v(t) = (1 + s_t)^{-t}$ facteur d'actualisation, f_t taux forward à un an, q_x probabilité de décès à l'âge x , BE best estimate, FPB fonds propres de base, D_m duration modifiée.

2 Package `scr_data` — fabrication des données

2.1 `config.py` — hypothèses et aléa

`charger_hypotheses(chemin)`

Point d'entrée unique vers le fichier d'hypothèses. Centraliser la lecture évite que chaque module relise le YAML avec ses propres conventions.

`_verifier(hyp)`

Refuse une configuration incohérente avant tout calcul. Trois contraintes de sommation, qui sont les erreurs de saisie les plus fréquentes : $\sum_c w_c = 1$ pour l'allocation, $\sum_q p_q = 1$ pour les répartitions par échelon de crédit, $\sum_{\text{pays}} w = 1$ pour les souverains.

`date_evaluation(hyp)`

Convertit la date en objet manipulable ; sert notamment à déterminer la génération de référence des tables de mortalité.

`generateur(hyp, decalage)`

Donne un générateur aléatoire par module, de graine graine + decalage. L'objectif est l'indépendance des blocs : modifier le passif ne doit pas déplacer les tirages de l'actif, sans quoi aucune comparaison avant/après n'est lisible.

`poids_normalises(n, rng, sigma)`

Répartit un montant sur n lignes de tailles hétérogènes mais réalistes, par tirage log-normal normalisé : $w_i = e^{\sigma Z_i} / \sum_j e^{\sigma Z_j}$, avec $Z_i \sim \mathcal{N}(0, 1)$.

2.2 `courbe.py` — courbe des taux sans risque

`_wilson(t, u, alpha, omega)`

Noyau de la méthode Smith-Wilson, brique de l'interpolation et de l'extrapolation réglementaires :

$$W(t, u) = e^{-\omega(t+u)} \left[\alpha \min(t, u) - \frac{1}{2} e^{-\alpha \max(t, u)} (e^{\alpha \min(t, u)} - e^{-\alpha \min(t, u)}) \right].$$

`SmithWilson.__init__(maturites, taux_pair, ufr, alpha)`

Calibre la courbe sur des swaps au pair, c'est-à-dire des instruments cotés à 100. Résout le système linéaire $(CWC^\top) \zeta = p - C\mu$ avec $\mu_t = e^{-\omega t}$, C la matrice des flux et $p = \mathbf{1}$.

`SmithWilson.facteur_actualisation(t)`

Donne le prix zéro-coupon à toute maturité, entière ou non : $P(t) = e^{-\omega t} + \sum_j \zeta_j W(t, u_j)$.

`SmithWilson.forward_instantane(t, h)`

Nécessaire au critère de convergence réglementaire. Différence finie centrée : $f(t) \simeq -(\ln P(t+h) - \ln P(t-h))/2h$.

`calibrer_alpha(...)`

Reproduit la règle EIOPA : retenir le plus petit $\alpha \geq 0,05$ tel que $|f(\text{CP}) - \ln(1 + \text{UFR})| \leq 1$ pb. Un α trop grand déformerait la partie liquide, un α trop petit ne convergerait pas.

`courbe_smith_wilson(hyp_courbe)`

Chaîne complète à partir de taux swap : retrait de l'ajustement pour risque de crédit ($s \rightarrow s - \text{CRA}$), calibrage, puis tabulation. Sert de repli quand le fichier officiel n'est pas disponible.

`courbe_eiopa(hyp_courbe)`

Lit la courbe publiée par l'EIOPA. Repère la cellule portant le nom de la devise, puis la première série de 150 valeurs numériques consécutives : ce repérage par contenu résiste aux changements de mise en page du classeur.

`_mettre_en_forme(t, df)`

Reconstruit les représentations utiles à partir des seuls facteurs d'actualisation : $s_t = P(t)^{-1/t} - 1$ et $f_t = P(t-1)/P(t) - 1$.

`construire_courbe(hyp)`

Aiguillage entre les deux sources, pour que tout le reste du code ignore d'où vient la courbe.

`Actualisation.df(t, spread)`

Actualisation à maturité quelconque, y compris non entière (flux de milieu d'année). Interpolation *logarithmique*, qui garantit des facteurs positifs et des forwards réguliers : $\ln P(t) = \text{interp}(t, \ln P)$, puis $P(t, \sigma) = (1 + s_t + \sigma)^{-t}$ en présence d'un spread.

2.3 mortalite.py — tables de mortalité

`TableMortalite.__init__(qx, nom)`

Encapsule un vecteur q_x borné à $[0, 1]$ avec $q_{120} = 1$, pour que toute projection se termine.

`TableMortalite.makeham(A, B, c)`

Table paramétrique de repli, utile pour travailler sans fichier réglementaire : $q_x = 1 - \exp(-A - Bc^x(c-1)/\ln c)$.

`TableMortalite.depuis_csv(chemin)`

Charge une table réglementaire au format age, qx.

`TableMortalite.ajuster(facteur)`

Matérialise les chocs de mortalité et de longévité : la table choquée est $q_x^{\text{choc}} = \lambda q_x$, avec $\lambda = 1,15$ ou $0,80$.

`TableMortalite.survie(age, n)`

Brique de toute valorisation viagère : ${}_k p_x = \prod_{j=0}^{k-1} (1 - q_{x+j})$.

`TableMortalite.annuite_viagere(age, actu, terme_echu, duree_max)`

Valeur actuelle probable d'une rente unitaire, à terme échu ou à terme à échoir :

$$a_x = \sum_{k=1} {}_k p_x v(k), \quad \ddot{a}_x = \sum_{k=0} {}_k p_x v(k).$$

`TableGenerationnelle.__init__(colonnes, annee_reference, nom, facteur)`

Les tables TGH05 et TGF05 sont bidimensionnelles : une personne d'âge x à la date Y relève de la génération $Y - x$. Cette classe évite l'erreur classique qui consiste à écraser la dimension générationnelle, laquelle sous-estimerait la longévité. L'attribut `qx` conserve la diagonale $q_x^{\text{diag}} = q(Y - x, x)$ pour les traitements à une dimension.

`TableGenerationnelle.depuis_csv_long(chemin, annee, nom)`

Lit le format generation, age, qx produit par le script d'extraction.

`TableGenerationnelle._colonne(generation)`

Renvoie la colonne demandée, ou la génération disponible la plus proche : les tables ne couvrent pas toutes les générations.

`TableGenerationnelle.survie(age, n)`

Survie le long de la génération : ${}_k p_x = \prod_{j < k} (1 - \lambda q(Y - x, x + j))$.

`charger_tables(hyp)`

Assemble le jeu de quatre tables (décès hommes et femmes, rentes hommes et femmes) selon la source configurée.

`_charger_rente(chemin, annee, nom)`

Détecte le format : la présence d'une colonne `generation` déclenche le traitement générationnel.

2.4 actifs.py — inventaire ligne à ligne

`prix_obligation(actu, coupon, maturite, spread)`

Valorisation de référence, utilisée à la génération comme sous choc :

$$P = 100 \left(c \sum_{t=1}^n v(t, \sigma) + v(n, \sigma) \right), \quad v(t, \sigma) = (1 + s_t + \sigma)^{-t}.$$

`duration_effective(actu, coupon, maturite, spread, choc)`

Mesure la sensibilité sur la courbe réelle plutôt que par une formule fermée, ce qui reste valable même si la courbe n'est pas plate :

$$D_m = \frac{P(\sigma - \varepsilon) - P(\sigma + \varepsilon)}{2P(\sigma)\varepsilon}, \quad \varepsilon = 1 \text{ pb.}$$

`_tirer_maturites(rng, n, moyenne, mmin, mmax)`

Produit un profil d'échéances dispersé mais centré : $M \sim \Gamma(2, \text{moyenne}/2)$, puis arrondi supérieur et troncature dans $[\text{mmin}, \text{mmax}]$.

`_coupon(rng, actu, maturite, spread, ecart)`

Donne des titres au-dessus et en dessous du pair, donc des plus et moins-values latentes réalistes : $c = s_n + \sigma + \mathcal{N}(-0,4\%, \text{ecart})$, arrondi au huitième de point.

`_lignes_obligataires(rng, actu, specs, montant_cible, hyp_obl)`

Valorise une liste de titres puis met les nominaux à l'échelle pour atteindre la valeur de marché visée : $N_i = w_i \text{cible} \times 100/P_i$.

`_souverains, _corporates, _actions, _immobilier, _monetaire, _prets_infra`

Un générateur par poche, parce que chaque poche porte des attributs différents et alimente des sous-modules différents : échelon et duration pour le spread, type d'action pour le choc action, devise pour le change, groupe d'émetteurs pour la concentration.

`_uc_transparence(rng, actu, hyp, montant)`

Met les supports en unités de compte en transparence. Sans cela, le risque porté par ces actifs serait invisible, alors que la réglementation impose de regarder au travers des OPC.

`generer_inventaire(hyp, actu)`

Assemble les poches, complète les colonnes manquantes et attribue les identifiants. C'est la seule fonction publique du module.

2.5 passif_vie.py — model points

`_ages, _sexes`

Tirages démographiques tronqués, pour éviter les âges aberrants qui casseraient les projections.

`_taux_par_tranche(anciennete, tranches, cle)`

Applique un barème par tranche d'ancienneté. Sert aux taux minimums garantis par génération de contrat et aux rachats structurels, dont le pic fiscal à huit ans.

`epargne_euro(hyp, rng)`

Construit le portefeuille central du modèle. Chaque model point porte ce dont la projection a besoin : PM, taux garanti, chargement, taux de rachat, âge.

`epargne_uc(hyp, rng)`

Même logique pour les unités de compte, avec les versements cumulés nécessaires à une éventuelle garantie plancher.

`temporaire_deces(hyp, tables, rng)`

Tarifie puis met à l'échelle, de sorte que le volume de primes corresponde au plan : $\pi = q_x \times C \times (1 + \text{chargement})$, puis $C \rightarrow C \times \Pi_{\text{cible}} / \sum \pi$.

`_annuite_reversion(t_princ, t_conj, age_x, age_y, actu)`

Valorise la réversion, versée au conjoint après le décès du rentier, sous hypothèse d'indépendance des durées de vie :

$$a_{\text{rev}} = \sum_{k \geq 1} v(k) {}_k p_y (1 - {}_k p_x).$$

`rentes_viageres(hyp, tables, actu, rng)`

Calibre les arrérages pour atteindre le best estimate cible : $R_i = \tilde{w}_i \times \text{BE}_{\text{cible}} / \sum_j \tilde{w}_j a_j$. Cette inversion garantit que le bilan reste cohérent quelle que soit la table utilisée.

`generer_passif_vie(hyp, tables, actu)`

Assemble les quatre tables de model points.

2.6 non_vie_sante.py — triangles et santé SLT

`_parts_cumulees(cadence, facteur_queue)`

Traduit une cadence de règlement en parts cumulées de l'ultime, en réservant la queue : $\pi_j = c_j / (c_{n-1} \times f_{\text{queue}})$.

`reserve_theorique(p, ...)`

Calcule la réserve attendue avant aléa, $\sum_i P_i \times S/P \times (1 - \pi_{n-1-i})$, ce qui permet de caler les paramètres avant toute simulation.

`simuler_triangle(nom, p, ...)`

Produit un triangle réaliste *et* les ultimes réels, ce qui permet de mesurer l'erreur des méthodes de provisionnement —

impossible sur données réelles. Ultime bruité en log-normal, incréments bruités en gamma :

$$U_i = P_i S / P \kappa e^{\mathcal{N}(-\frac{\varsigma^2}{2}, \varsigma^2)}, \quad X_{i,j} = U_i(\pi_j - \pi_{j-1}) G, \quad G \sim \Gamma(k, 1/k), \quad k = 1/CV^2.$$

`triangle_en_matrice(tri_long)`

Bascule du format long au format matriciel attendu par Chain Ladder et par l'export.

`generer_non_vie(hyp)`

Boucle sur les lignes d'activité et assemble triangles, ultimes, volumes et expositions.

`_volume(...)`

Prépare les volumes du sous-module primes et réserves : P_{last} , P_{next} , primes futures, taux de cession.

`_expositions_cat(hyp, rng)`

Répartit les capitaux assurés par zone selon une loi de Dirichlet, ce qui crée une concentration géographique réaliste, et rassemble les autres expositions catastrophe.

`generer_sante_slit(hyp, tables, actu)`

Rentes d'incapacité et d'invalidité en cours de service, avec deux lois de maintien distinctes : mortalité aggravée jusqu'à la fin de garantie pour l'invalidité, maintien exponentiel $e^{-\lambda t}$ pour l'incapacité. Les rentes sont ensuite calées sur le best estimate cible.

2.7 bilan.py et export.py

`generer_contreparties(hyp)`

Construit les expositions de type 1 et 2, seule entrée du module de défaut.

`construire_bilan(...)`

Présente le bilan au format proche du S.02.01 et, surtout, marque chaque poste par son statut : issu de l'inventaire, hypothèse, indicatif ou cible à recalculer. Sans ce marquage, on oublie vite ce qui est calculé et ce qui est posé.

`controles(...)`

Vingt et un contrôles de calibrage. Leur rôle est d'échouer tôt : un écart d'allocation ou un calage de triangle aberrant se voit immédiatement, avant toute interprétation des SCR.

`exporter_csv, _entete, _style, _titre, exporter_excel`

Restitution. Les totaux du classeur sont écrits en formules Excel et non en valeurs, pour qu'un lecteur puisse modifier une ligne et voir le total suivre.

3 generer_donnees.py

`generer(chemin_config, ecrire)`

Orchestre la fabrication complète des données et garantit l'ordre des dépendances : courbe, puis tables, puis actifs, puis passifs.

`_resume(...)`

Affiche les masses, les fonds propres et le décompte des contrôles, en remontant les avertissements de calage capturés pendant la génération.

4 Package scr — moteur de calcul

4.1 passif.py — best estimate déterministe

Chocs

Dataclass gelée portant tous les chocs de passif. Gelée pour deux raisons : elle est comparable par égalité, ce qui permet au module marché de détecter qu'un choc ne touche que les unités de compte et d'emprunter un raccourci de calcul ; et elle ne peut pas être modifiée par inadvertance au milieu d'une projection.

`_qx_projete(table, age, horizon, chocs)`

Trajectoire de mortalité sous choc : $q_{x+k}^{\text{choc}} = \lambda q_{x+k}$, et $q_x^{\text{choc}} = q_x + 0,0015$ la première année pour le choc catastrophe (art. 144).

`_frais_unitaires(base, hyp, chocs, horizon)`

Frais inflatés, avec le point d'inflation supplémentaire du choc de frais (art. 143) : $g_k = g_0(1 + \lambda_f)(1 + i + \Delta i)^k$.

`be_epargne_euro(mp, actu, courbe, tables, hyp, chocs, par_model_point)`

Cœur du passif vie. Projection annuelle par model point :

$$\begin{aligned} r_k &= \max(\text{TMG} + \text{chargement}, \alpha f_k) - \text{chargement} \\ \text{PM}_{k+1} &= \text{PM}_k(1 + r_k)(1 - q_k - \ell_k + q_k \ell_k) \\ \text{BE} &= \sum_k v(k) \left[\text{PM}_k(1 + r_k)(q_k + \ell_k - q_k \ell_k) + g_k \text{PM}_k \right] \end{aligned}$$

L'option `par_model_point` renvoie le détail contrat par contrat, indispensable au rachat massif sélectif.

`be_epargne_uc(...)`

Même structure, mais la provision croît au forward diminué des prélèvements, ce qui fait ressortir naturellement $\text{BE} < \text{PM}$: la différence est la valeur actuelle des chargements futurs.

`be_temporaire_deces(mp, actu, tables, hyp, chocs)`

Garantie décès, nette des primes futures — d'où un best estimate parfois négatif :

$$\text{BE} = \sum_k {}_k p_x^{(\tau)} q_{x+k} C v(k + \tfrac{1}{2}) + \gamma \sum_k {}_k p_x^{(\tau)} \pi v(k) - \sum_k {}_k p_x^{(\tau)} \pi v(k),$$

le décrétement ${}_k p^{(\tau)}$ combinant décès et chute.

`be_rentes(mp, actu, tables, hyp, chocs)`

Valorise les rentes en service, réversion comprise : $\text{BE} = \sum_i R_i(1 + \varphi) [\ddot{a}_{x_i} + \tau a_{\text{rev}}]$.

`be_sante_slts(mp, actu, tables, hyp, chocs)`

Rentes d'incapacité et d'invalidité, avec les deux lois de maintien correspondantes et le choc de révision (art. 158).

`chain_ladder(triangle)`

Méthode de provisionnement de référence :

$$\hat{f}_j = \frac{\sum_{i \leq n-j-1} C_{i,j+1}}{\sum_{i \leq n-j-1} C_{i,j}}, \quad \hat{C}_{i,n} = C_{i,n-i} \prod_{j \geq n-i} \hat{f}_j, \quad \hat{R}_i = \hat{C}_{i,n} - C_{i,n-i}.$$

`cadence_reglement(facteurs)`

Déduit la cadence de règlement des facteurs de développement, pour pouvoir actualiser : $\pi_j = \prod_{k < j} \hat{f}_k / \prod_k \hat{f}_k$.

`be_non_vie(donnees, actu, hyp, chocs, modules)`

Transforme des réserves comptables en best estimate, c'est-à-dire actualisées et majorées des frais de gestion :

$$\text{BE}_{\text{sin}} = (1 + \gamma) \sum_i \hat{R}_i \sum_j \frac{\pi_{j+1} - \pi_j}{1 - \pi_{\text{dev}(i)}} v(j + \tfrac{1}{2}), \quad \text{BE}_{\text{prim}} = \text{exposition} \times \text{CR} \times v(1).$$

`Passifs.evaluer(actu, courbe, chocs)`

Interface unique du passif : tous les modules de risque passent par elle, ce qui garantit que le même jeu d'hypothèses est appliqué partout.

4.2 marche.py — risque de marché

`_facteurs_officiels(chemin)`

Lit l'onglet des chocs du fichier EIOPA, avec mise en cache : les facteurs sont relus à chaque évaluation de sous-module.

`facteurs_choc_taux(params, sens, maturites)`

Fournit $f_t^\uparrow$ et $f_t^\downarrow$, soit depuis le fichier officiel, soit depuis les tables du paramétrage, avec interpolation linéaire entre les points tabulés.

`courbe_choquee(courbe, params, sens)`

Applique la règle des articles 166 et 167, exactement comme le classeur EIOPA :

$$s_t^\uparrow = s_t + \max(1\%, f_t^\uparrow |s_t|), \quad s_t^\downarrow = \begin{cases} s_t & s_t < 0 \\ s_t - f_t^\downarrow |s_t| & \text{sinon.} \end{cases}$$

`revaloriser_obligations(inv, actu)`

Reprix chaque titre sous une courbe donnée, en conservant coupon, maturité et spread. C'est ce qui rend le choc de taux exact plutôt qu'approché par la duration.

`facteur_stress_spread(cqs, duration, params, covered)`

Table de l'article 176, avec plancher de duration à un an et traitement réduit des obligations sécurisées : $\text{stress} = \min(a_{q,k} + b_{q,k}(d - d_k), 1)$.

`matrice_marche(matrice, a, b)`

Substitue les symboles de la matrice : A pour le couple taux-action-immobilier, B pour le couple taux-spread du régime 2027. Écrire la matrice avec des symboles évite de la dupliquer pour chaque scénario.

`MoteurMarche.__init__(...)`

Mémoire l'état de référence — best estimate et fonds propres avant choc — puisque chaque sous-module se mesure par rapport à lui.

`MoteurMarche._bof(valeurs, courbe, chocs)`

Recalcule les fonds propres de base sous un jeu de valeurs d'actif : $\text{FPB} = \sum_i \text{VM}_i + A_{\text{hors inventaire}} - \text{BE} - \text{autres passifs}$. Contient un raccourci : si seule la valeur des unités de compte bouge, seul leur best estimate est reprojeté, ce qui divise le temps de calcul.

`MoteurMarche._perte(...)`

Traduit la variation en SCR : $\max(0, \text{FPB}_0 - \text{FPB}_{\text{choc}})$. Le plancher à zéro interdit qu'un choc favorable réduise le capital requis.

`MoteurMarche._facteur_uc(valeurs)`

Transmet au passif la variation de valeur des supports en unités de compte : le porteur supporte le risque, donc la provision suit l'actif.

`MoteurMarche.taux()`

Évalue les deux scénarios et retient le pire, en mémorisant lequel : le choix détermine ensuite le paramètre de corrélation A .

`MoteurMarche.action()`

Chocs par catégorie (art. 168 à 172) puis agrégation : $\sqrt{\text{SCR}_1^2 + 2\rho \text{SCR}_1 \text{SCR}_2 + \text{SCR}_2^2}$, $\rho = 0,75$.

`MoteurMarche.immobilier()`

Choc uniforme de 25 % (art. 174), immeubles d'exploitation compris.

`MoteurMarche.spread()`

Applique le stress ligne à ligne, en excluant les souverains de l'Espace économique européen et en réduisant l'infrastructure qualifiante.

`MoteurMarche.change()`

Teste $\pm 25\%$ par devise (art. 188) et retient le pire sens, devise par devise.

`MoteurMarche.concentration()`

Ne sanctionne que l'excès au-delà d'un seuil (art. 184) :

$$\text{XS}_i = \max\left(0, \frac{E_i}{A} - \text{CT}_q\right), \quad \text{Conc}_i = A \text{XS}_i g_q, \quad \text{SCR} = \sqrt{\sum_i \text{Conc}_i^2}.$$

`MoteurMarche.calculer()`

Agrège les six sous-modules avec le paramètre A cohérent avec le scénario de taux retenu.

4.3 contrepartie.py

`_pd_exposition(cqs, params)`

Traduit un échelon de qualité de crédit en probabilité de défaut, avec repli pour les expositions non notées.

`scr_type1(expositions, params)`

Formule de variance de l'article 200, qui distingue le risque systématique du risque idiosyncrasique :

$$V = \sum_{j,k} \frac{\text{PD}_j \text{PD}_k}{1,25(\text{PD}_j + \text{PD}_k) - \text{PD}_j \text{PD}_k} \text{TLGD}_j \text{TLGD}_k + \sum_j \frac{1,5 \text{PD}_j (1 - \text{PD}_j)}{2,5 - \text{PD}_j} \sum_{i \in j} \text{LGD}_i^2,$$

puis règle à trois branches : $3\sqrt{V}$, $5\sqrt{V}$ ou $\sum \text{LGD}$ selon la concentration.

`scr_type2(contreparties, params)`

Traitement forfaitaire des créances : $15\% \times \text{non échues} + 90\% \times \text{échues depuis plus de trois mois}$.

`calculer(contreparties, params)`

Construit les pertes en cas de défaut — 50 % pour la réassurance, 100 % pour la trésorerie — et agrège : $\sqrt{SCR_1^2 + 1,5 SCR_1 SCR_2 + SCR_2^2}$

4.4 souscription.py — vie et santé

`agreger(scr, correlations)`

Forme quadratique commune à tous les modules : $\sqrt{\mathbf{v}^\top \mathbf{C} \mathbf{v}}$. Une seule implémentation, donc une seule occasion de se tromper.

`MoteurVie._evaluer(chocs) / _evaluer_mp(chocs)`

Best estimate des quatre segments vie, globalement ou par model point.

`MoteurVie._perte(chocs, segments)`

Retient la hausse de provisions *segment par segment* : $\sum_s \max(0, BE_s^{\text{choc}} - BE_s)$. Un segment où le choc est favorable ne vient pas compenser, conformément à l'esprit du texte.

`MoteurVie.mortalite() / longevite()`

Chocs permanents $\times 1,15$ (art. 137) et $\times 0,80$ (art. 138).

`MoteurVie.rachat()`

Trois variantes — hausse, baisse, rachat massif — dont la pire est retenue (art. 142).

`MoteurVie._rachat_massif()`

Applique le rachat de 40 % *contrat par contrat*, en ne retenant que ceux dont le rachat accroît le best estimate : $\sum_i \max(0, BE_i^{\text{massif}} - BE_i)$. Une application globale sous-estimerait le choc.

`MoteurVie.frais() / revision() / catastrophe()`

+10 % de frais et un point d'inflation (art. 143) ; révision des rentes (art. 141) ; surmortalité de 0,15 point la première année (art. 144).

`MoteurSante.slt()`

Transpose la logique vie aux rentes en service, en remplaçant le rachat par le risque d'invalidité : les taux de sortie d'incapacité sont réduits de 20 %.

`MoteurSante.nslt()`

Applique la formule primes et réserves à la ligne frais de soins :

$$\sigma_s = \frac{\sqrt{(\sigma_p V_p)^2 + \sigma_p V_p \sigma_r V_r + (\sigma_r V_r)^2}}{V_p + V_r}, \quad SCR = 3 \sigma_s V_s.$$

`MoteurSante.catastrophe()`

Version paramétrique simplifiée des articles 160 à 163 : accident de masse, concentration d'accident et pandémie, agrégés quadratiquement. Assumée comme provisoire, elle isole le chantier restant.

4.5 non_vie.py

`MoteurNonVie.primes_reserves()`

Écart type par ligne puis agrégation par la matrice de l'annexe IV :

$$\sigma = \frac{\sqrt{\sum_{s,t} \text{Corr}_{s,t} \sigma_s V_s \sigma_t V_t}}{V}, \quad SCR = 3 \sigma V.$$

Le facteur 3 est l'approximation log-normale du quantile à 99,5 %.

`MoteurNonVie.rachat()`

Ne charge du capital que si la provision pour primes est bénéficiaire : $40\% \times \max(0, \text{exposition} - BE_{\text{primes}})$.

`MoteurNonVie.catastrophe()`

Agrège périls naturels et scénarios d'origine humaine, puis applique la réassurance.

`MoteurNonVie._appliquer_xl(brut)`

Traité en excédent de sinistres : $L^{\text{net}} = \min(L, \text{priorité}) + \max(0, L - \text{priorité} - \text{portée})$. Appliqué avant agrégation, sinon l'effet sur la queue serait mal capté.

4.6 agregation.py

`bscr(scr_modules, params)`

Agrégation des cinq modules (art. 87) : $BSCR = \sqrt{\sum_{i,j} \text{Corr}_{i,j} SCR_i SCR_j}$.

`operationnel(hyp, be, bscr, params, donnees)`

Formule forfaitaire (art. 204) :

$$SCR_{op} = \min(0,3 BSCR, \max(Op_{prim}, Op_{prov})) + 0,25 Dp_{UC}.$$

`ajustements(bscr, scr_op, hyp, params)`

Calcule les deux ajustements. La capacité fiscale est plafonnée deux fois : par le taux d'impôt appliqué au choc, et par la capacité réellement justifiable (art. 207).

`mcr(scr, be, donnees, hyp, params, detail_non_vie)`

Calcul linéaire puis encadrement (art. 248 à 253) : $MCR = \max\{\min(\max(MCR_{lin}, 0,25 SCR), 0,45 SCR), AMCR\}$.

`tableau_s25(...)`

Restitue le résultat au format réglementaire, la diversification étant présentée comme l'écart entre le BSCR et la somme des modules.

4.7 esg.py — générateur de scénarios

`HullWhite.__init__(courbe, a, sigma)`

Extrait le forward instantané initial, condition nécessaire pour que le modèle reproduise exactement la courbe de marché.

`HullWhite.alpha(t)`

Terme d'ajustement qui assure cette reproduction : $\alpha(t) = f(0,t) + \frac{\sigma^2}{2a^2} (1 - e^{-at})^2$.

`HullWhite.prix_zc(t, maturite, r)`

Structure affine du modèle : $P(t,T) = A(t,T)e^{-B(t,T)r_t}$ avec $B(t,T) = (1 - e^{-a(T-t)})/a$. Donne le taux à dix ans conditionnel, dont le modèle ALM a besoin.

`HullWhite.simuler(...)`

Schéma *exact* à pas annuel, donc sans biais de discrétisation :

$$r_{t+1} = r_t e^{-a} + \alpha(t+1) - \alpha(t) e^{-a} + \sigma \sqrt{\frac{1-e^{-2a}}{2a}} \varepsilon_{t+1}.$$

`_bruits_correles(...)`

Corrèle les trois facteurs par décomposition de Cholesky et double les tirages en variables antithétiques, ce qui réduit la variance sans biais.

`generer_scenarios(courbe, hyp)`

Assemble taux, déflateurs et rendements risqués : $D_t = \exp(-\int_0^t r_u du)$ approché par la règle du trapèze, et $S_{t+1}/S_t = \exp(r_t - \frac{\sigma_S^2}{2} + \sigma_S \varepsilon)$.

`scenario_central(courbe, hyp)`

Scénario déterministe fondé sur les forwards. Sans lui, la valeur temps des options ne serait pas isolable.

`tests_martingale(sc)`

Contrôle de qualité indispensable : $\mathbb{E}[D_T] = P(0,T)$ et $\mathbb{E}[D_T S_T]/S_0 = 1$. Un générateur qui échoue ici produit des best estimate faux.

4.8 alm.py — projection actif-passif

`construire_cohortes(mp, hyp, tables, horizon)`

Regroupe les model points par taux garanti et tranche d'âge. La vectorisation par cohortes est ce qui rend praticables des milliers de scénarios.

`construire_actif_euro(inventaire, hyp)`

Agrège l'inventaire en quatre poches avec leurs plus-values latentes, leur rendement courant et leur duration.

`_taux_rachat_conjoncturel(ecart, lois)`

Loi ONC de l'ACPR, fonction affine par morceaux de l'écart entre taux servi et taux attendu, à mi-chemin des courbes plancher et plafond. C'est le mécanisme qui relie le comportement des assurés à l'environnement de taux.

`ModeleALM.projeter(sc, regime, ...)`

Boucle annuelle vectorisée sur les scénarios. Points clés :

- rendement comptable renouvelé progressivement : $y_{t+1} = y_t(1 - 1/M) + s_t^{10}/M$;
- réalisation de plus-values pour atteindre la cible : $\text{réal} = \min(\theta \text{PVL}, \text{besoin})$;
- participation aux bénéfices, dotation et reprise obligatoire de la PPE sur huit ans ;
- best estimate = $\mathbb{E}[\sum_t D_t(\text{prestations}_t + \text{frais}_t)]$.

Les trois régimes — dynamique, figé, garanti — donnent respectivement le calcul net, le calcul brut et la part garantie, dont la différence mesure les prestations discrétionnaires futures.

4.9 reforme.py — régime 2027

`dernier_forward_liquide(courbe, fsp, poids)`

Calcule le LLFR, moyenne pondérée des forwards liquides au-delà du premier point de lissage. C'est le pivot de la nouvelle extrapolation.

`extrapoler_2027(courbe, params, ufr, decalage)`

Nouvelle méthode :

$$f(h) = \text{LLFR} + (\text{UFR}^c - \text{LLFR}) \frac{1 - e^{-ah}}{ah},$$

appliquée au-delà du premier point de lissage, avec UFR décalée sous choc.

`courbe_choquee_2027(courbe, params, sens, ufr)`

Choc multiplicatif plus décalage parallèle, avec planchers négatifs par terme, puis ré-extrapolation : $s_t^\uparrow = s_t(1 + b_t) + a_t$ et $s_t^\downarrow = \max(s_t(1 - b_t) - a_t, s_t)$.

`params_2027(params, reforme, etapes)`

Applique les modifications *une à une*, sur une copie profonde. C'est ce qui rend possible le pont du ratio, étape par étape, et garantit que le paramétrage d'origine n'est jamais altéré.

4.10 marge_risque.py

`profil_run_off(flux, actu)`

Décroissance des provisions restant à couvrir, normalisée à 1. Sert d'inducteur aux SCR futurs, conformément aux simplifications admises.

`marge_risque(scr_initial, flux, actu, cout_capital, lambda, plancher)`

Formule de la marge de risque, dans les deux régimes :

$$\text{MR} = \text{CoC} \sum_{t \geq 0} \frac{\max(\lambda^t, \underline{\lambda}) \text{SCR}^{\text{nc}}(t)}{(1 + s_{t+1})^{t+1}}, \quad \text{SCR}^{\text{nc}}(t) = \text{SCR}^{\text{nc}}(0) \frac{\text{BE}(t)}{\text{BE}(0)}.$$

Le facteur λ n'existe que dans le régime 2027.

4.11 modele_interne.py

`bootstrap_odp(triangle, nb_simulations, rng, ajustement_biais)`

Donne la distribution complète des réserves, là où Chain Ladder ne donne qu'une espérance. Résidus de Pearson corrigés du biais, puis bruit de processus gamma :

$$r_{i,j} = \frac{X_{i,j} - \hat{m}_{i,j}}{\sqrt{\hat{m}_{i,j}}} \sqrt{\frac{N}{N-p}}, \quad \hat{\phi} = \frac{\sum r^2}{N-p}, \quad X^* \sim \Gamma(\hat{m}/\hat{\phi}, \hat{\phi}).$$

`ajuster_sp(triangle, primes)`

Estime la loi des sinistres à primes ultimes sur l'historique : μ, ς des $\ln(\hat{U}_i/P_i)$.

`simuler_primes(mu, sigma, volume, n, rng, incertitude)`

Charge de l'année à venir, $L = V \exp(\mu + \varsigma(1 + \eta)Z)$, la majoration η représentant l'incertitude d'estimation sur dix observations seulement.

`simuler_catastrophe(params, n, rng, priorite, portee)`

Fréquence-sévérité au-delà d'un seuil, qui est la modélisation naturelle des extrêmes : $N \sim \mathcal{P}(\lambda)$ et $X = u + \frac{\beta}{\xi}((1 - U)^{-\xi} - 1)$, avec application du traité par événement.

`copule(matrice, n, rng, famille, ddl)`

Sépare les marginales de la structure de dépendance. La copule de Student, contrairement à la gaussienne, produit des chocs simultanés dans les queues : $U = t_v(Z\sqrt{v/W})$, $W \sim \chi_v^2$.

`coupler(marginales, uniformes)`

Relie des marginales simulées séparément par les quantiles empiriques : $X_k = F_k^{-1}(U_k)$. Les marginales peuvent avoir des tailles différentes.

`mesures_risque(pertes, quantile)`

Mesures de risque comparables : $\text{SCR} = \text{VaR}_{99,5\%}(L) - \mathbb{E}[L]$ et $\text{TVaR} = \mathbb{E}[L \mid L \geq \text{VaR}]$.

`usp_reserve(distribution, sigma_standard, nb_annees, credibilite)`

Paramètre propre à l'entreprise, mélangé au paramètre de marché par crédibilité : $\sigma^* = c\sigma^{\text{USP}} + (1 - c)\sigma^{\text{std}}$, avec c fonction de la profondeur d'historique.

4.12 orsa.py

`EtatBilan`

Porte l'état du bilan projeté et ses propriétés dérivées. Le fait que $\text{FP} = A - P$ soit une propriété calculée, et non un champ, interdit les incohérences.

`etat_initial(be_segments, hyp, inventaire)`

Construit le point de départ à partir du bilan V2 et de l'allocation réelle.

`ProjectionORSA.scr_initial()`

Contrôle de cohérence : reconstitue le SCR de départ avec les briques de la projection. Il a détecté un double comptage de l'absorption.

`ProjectionORSA._modules_projetes(...)`

Met les modules à l'échelle de leurs inducteurs de volume, plutôt que de recalculer un SCR complet chaque année : c'est la simplification usuelle en ORSA. Le module marché réagit en outre à la déformation de l'allocation, à hauteur de 30 %.

`ProjectionORSA.projeter(scenario)`

Boucle annuelle : environnement financier et chocs, activité, compte de résultat, bouclage du bilan. Le résultat s'écrit

$$R = rA_{\text{début}} - \text{taux servi} \times \text{BE}_{\text{début}}^{\text{euro}} + \text{marges} - \text{frais} - \text{sinistres},$$

et les fonds propres évoluent de R net d'impôt et de dividende, l'actif se déduisant du passif. Ce bouclage rend l'égalité comptable vraie par construction.

`besoin_global_solvabilite(scr, hyp)`

Complète le SCR par les risques mal captés par la formule standard, avec abattement de diversification. C'est le cœur du message ORSA.

5 Scripts d'extraction

`extraire_tables_mortalite._qx_depuis_lx(lx)`

Les classeurs officiels donnent des effectifs, les modèles ont besoin de probabilités : $q_x = 1 - l_{x+1}/l_x$.

`extraire_tables_mortalite.extraire_th_tf(chemin)`

Isole les quatre tables du classeur TH et TF, y compris les variantes en cas de vie, souvent oubliées.

`extraire_tables_mortalite.extraire_generationnelle(chemin, onglet)`

Met les tables TGH05 et TGF05 au format long generation, age, qx, seul format qui préserve la dimension générationnelle.

`extraire_tables_mortalite.diagonale(table, annee)`

Produit une table à une dimension pour les outils qui n'en acceptent pas d'autre : $q_x^{\text{diag}} = q(Y - x, x)$.

`extraire_parametres_eiopa.facteurs_choc(chemin)`

Extrait les facteurs $f^{\uparrow}$ et $f^{\downarrow}$ officiels, ce qui supprime une saisie manuelle et donc une source d'erreur.

`extraire_parametres_eiopa.parametres(chemin, devise)`

Récupère UFR, dernier point liquide, point de convergence, α et ajustement pour risque de crédit, qui documentent la construction de la courbe.

6 Scripts de calcul

`calculer_scr.charger_donnees(dossier)`

Lit les tables et échoue avec un message explicite si la génération n'a pas été lancée.

`calculer_scr.calculer(ecrire, hyp, params, courbe)`

Enchaîne les modules jusqu'au S.25.01. Les trois arguments optionnels sont ce qui permet aux versions suivantes de rejouer le calcul avec d'autres hypothèses, sans dupliquer une ligne de code.

`calculer_v2._stress_spread_moyen(inventaire, params)`

Résume le choc de spread en une perte relative moyenne, transmissible au modèle ALM qui travaille sur des poches agrégées.

`calculer_v2.substituer(perte_v1, choc, regime)`

Cœur de la V2 :

$$\text{perte}^{V2} = \text{perte}^{V1} - \Delta BE_{\text{euro}}^{\text{déterministe}} + \Delta BE_{\text{euro}}^{\text{stochastique}}.$$

Cette écriture remplace proprement le canton euro dans chaque sous-module, sans réécrire les modules eux-mêmes.

`calculer_v2._rachat_massif_selectif(...)`

Applique le rachat massif cohorte par cohorte, avec PPE allouée au prorata, pour préserver la sélection exigée par l'article 142.

`calculer_v2._sensibilite_tmg(...)`

Mesure la valeur temps des options à quatre niveaux de taux garantis. C'est ce qui révèle sa forme en cloche.

`calculer_v3.scr_non_couvrable(resultat, params)`

Assiette correcte de la marge de risque : agrégation des modules en annulant le risque de marché, plus le risque opérationnel.

`calculer_v3.calculer()`

Rejoue la V2 à chaque étape du pont, en remplaçant courbe et paramètres selon l'ensemble cumulé des modifications.

`calculer_v3._sensibilites(...)`

Isole les deux paramètres les plus incertains : la part du décalage parallèle des chocs de taux, et le niveau de l'ajustement symétrique.

`calculer_v4._recalculer_global(v2, scr_non_vie, hyp, params)`

Substitue le module non-vie dans l'agrégation et recalcule les ajustements, pour mesurer l'effet d'un modèle interne sur le ratio et non seulement sur un module.

`calculer_v4._marge_cima(...)`

Calcule la marge de solvabilité du code CIMA sur le même bilan : $\max(20\% P, 25\% S)$ en non-vie, $5\% PM + 0,3\% C$ en vie. La comparaison montre ce qu'un référentiel forfaitaire ne voit pas.

`calculer_v5._stress_inverses(projection, hyp)`

Recherche par dichotomie l'amplitude de choc qui amène le ratio à la limite d'appétence. Le stress inversé répond à la question que pose un conseil d'administration : jusqu'où peut-on aller ?

`calculer_v5._synthese(...)`

Résume chaque scénario par son ratio minimal, l'année du creux, les franchissements de seuils et le capital à injecter.

7 Restitution

`scr/rapport.ecrire_rapport(...)`

Classeur de restitution : S.25.01, détail par module, MCR et best estimate.

`rapport/generer_rapport.macros()` et `tableaux()`

Écrivent les valeurs et les corps de tableaux sous forme de macros LaTeX. L'objectif est qu'aucun chiffre du rapport ne soit recopié à la main : relancer les modèles suffit à le mettre à jour.

`rapport/generer_rapport.nb()`, `pourcent()`, `texte()`

Formatage à la française et échappement des caractères réservés. Une seule fonction d'échappement évite les tableaux cassés par un signe pourcent non protégé.

`pedagogie/generer_chiffres.bloc_*`

Un bloc par module, qui refait les étapes intermédiaires — actualisation flux par flux, facteur de stress, forme quadratique, formule de variance — pour que le cahier pédagogique montre le détail du calcul et non seulement son résultat.

8 Index : article du règlement, fonction correspondante

Article	Fonction
28, 77	<code>be_*, Passifs.evaluer</code>
37 à 39	<code>marge_risque</code>
87, 103	<code>agregation.bscr</code>
114 à 118	<code>MoteurNonVie.primes_reserves, .rachat</code>
119 à 135	<code>MoteurNonVie.catastrophe</code>
136 à 144	<code>MoteurVie.*</code>
144 à 163	<code>MoteurSante.*</code>
164	<code>MoteurMarche.calculer, matrice_marche</code>
166, 167	<code>courbe_choquee, facteurs_choc_taux</code>
168 à 173	<code>MoteurMarche.action</code>
174	<code>MoteurMarche.immobilier</code>
176, 180	<code>facteur_stress_spread</code>
184 à 187	<code>MoteurMarche.concentration</code>
188	<code>MoteurMarche.change</code>
189 à 202	<code>contrepartie.scr_type1, .scr_type2</code>
204	<code>agregation.operationnel</code>
207	<code>agregation.ajustements</code>
218, annexe XVII	<code>usp_reserve</code>
248 à 253	<code>agregation.mcr</code>
Directive 2025/2	<code>reforme.*, marge_risque (CoC 4,75 %)</code>